

Project Proposals: Network Analysis of AI Coding Tool Adoption on GitHub

Alexander Quispe
CS 145: Networks
California Institute of Technology

March 2026

Abstract

We present multiple project proposals analyzing the adoption and diffusion of AI coding assistants (Claude Code, GitHub Copilot, OpenAI Codex, and Google Jules) across the GitHub developer network. Using a novel dataset of 5+ million AI-assisted code contributions spanning 600,000+ repositories, we propose investigating cascade effects, social influence, and network dynamics in technology adoption. Each proposal includes motivation, methodology, network science foundations, and expected deliverables.

1 Introduction and Dataset Overview

The emergence of AI coding assistants represents one of the most significant technological shifts in software development history. Between February 2025 and January 2026, we collected comprehensive data on four major AI coding tools:

Table 1: Dataset Summary: AI Coding Tool Usage on GitHub

Tool	Events	Repositories	Users (est.)	Detection Method
Claude Code	281,058	438,979	50,000+	Co-author trailers
GitHub Copilot	1,082,564	246,957	200,000+	Branch prefix <code>copilot/</code>
OpenAI Codex	3,094,740	248,608	300,000+	Branch prefix <code>codex/</code>
Google Jules	622,121	70,116	20,000+	Bot commits
Total	5,080,483	~600,000	500,000+	—

This dataset enables unprecedented analysis of technology adoption dynamics in developer networks. Below we present six distinct project proposals, each offering a unique perspective on network effects in AI tool adoption.

2 Project Proposal 1: Cascade Effects in AI Tool Adoption

2.1 The Idea

Core Question

Does AI coding tool adoption spread through developer collaboration networks like an information cascade, and can we predict which developers will adopt based on their network position?

We propose modeling AI tool adoption as an **information cascade** through GitHub’s developer collaboration network. When a developer sees their collaborators successfully using Claude Code or Copilot, they may be more likely to adopt the tool themselves. This creates cascade dynamics similar to those studied in viral marketing and social contagion.

2.1.1 Network Construction

We will construct a **developer collaboration network** $G = (V, E)$ where:

- V = set of developers (identified by GitHub username)
- E = edges connecting developers who have contributed to the same repository
- Edge weights w_{ij} = number of shared repositories between developers i and j

2.1.2 Cascade Model

We adapt the **Independent Cascade Model** for AI tool adoption:

1. At time $t = 0$, a seed set S_0 of “early adopters” has adopted tool T
2. At each time step t , newly adopted developers attempt to “infect” their neighbors
3. Developer j adopts from neighbor i with probability:

$$p_{ij} = \frac{w_{ij}}{\sum_k w_{ik}} \cdot \beta$$

where β is the base transmission rate (to be estimated)

2.1.3 Temporal Analysis

Using our timestamped data, we can:

1. Identify when each developer first adopted each tool
2. Reconstruct the temporal sequence of adoptions
3. Test whether adoption follows network structure

2.2 Motivation

Understanding cascade dynamics has significant implications:

1. **For AI companies:** Identify influential developers who can seed adoption
2. **For researchers:** Understand how new technologies diffuse through professional networks
3. **For organizations:** Predict which teams will adopt AI tools based on network position

The network perspective goes beyond simple adoption curves by explaining *why* certain developers adopt faster than others.

2.3 Methodology

Algorithm 1 Cascade Detection in AI Tool Adoption

- 1: **Input:** Commits/PRs with timestamps, repository contributors
 - 2: **Output:** Cascade trees, influence parameters
 - 3:
 - 4: Build collaboration graph G from shared repository contributions
 - 5: For each tool $T \in \{\text{Claude, Copilot, Codex, Jules}\}$:
 - 6: Extract adoption timestamps $\{(u_i, t_i)\}$ for each user
 - 7: Sort adoptions chronologically
 - 8: For each adoption (u_j, t_j) :
 - 9: Find neighbors $N(u_j)$ who adopted before t_j
 - 10: Assign most likely “parent” based on timing and edge weight
 - 11: Construct cascade tree
 - 12: Estimate transmission parameters via maximum likelihood
 - 13: Validate on held-out data (predict future adoptions)
-

2.4 Expected End Product

1. **Visualization:** Interactive cascade trees showing how adoption spread through the network
2. **Model:** Calibrated Independent Cascade model with estimated parameters
3. **Predictions:** Identify developers likely to adopt in the next 6 months
4. **Paper:** Analysis comparing cascade dynamics across the four AI tools

3 Project Proposal 2: Social Influence vs. Homophily in Tool Selection

3.1 The Idea

Core Question

Do developers adopt AI tools because their collaborators influenced them (social influence), or because similar developers independently make similar choices (homophily)?

This is the classic **identification problem** in network analysis: distinguishing causation from correlation. If two connected developers both use Claude Code, is it because:

- **Influence:** One developer saw the other using it and was convinced to try it?
- **Homophily:** Both developers share characteristics (e.g., Python expertise) that independently predict Claude adoption?
- **Confounding:** Both developers work at the same company with an enterprise Claude license?

3.2 Motivation

This distinction has profound implications:

- If **influence dominates**: Targeting influential developers can accelerate adoption
- If **homophily dominates**: Marketing should focus on developer characteristics, not network position
- Understanding the mechanism helps predict future technology adoption patterns

3.3 Methodology

We will apply several identification strategies:

3.3.1 Temporal Analysis (Granger-style)

If influence is real, adoption by neighbors at time $t - 1$ should predict adoption at time t , even after controlling for individual characteristics:

$$\text{Adopt}_{i,t} = \alpha + \beta \cdot \text{NeighborAdopt}_{i,t-1} + \gamma \cdot X_i + \epsilon_{i,t}$$

where X_i includes programming language, repository topics, geographic location, and account age.

3.3.2 Matched Pair Analysis

For each developer who adopted after a neighbor, find a “matched” developer with:

- Same programming language profile
- Similar repository count
- Similar follower count
- BUT no neighbors who adopted

Compare adoption rates between matched and exposed developers.

3.3.3 Network Randomization Tests

Shuffle adoption times while preserving:

1. Network structure
2. Total number of adoptions per time period
3. Individual developer characteristics

If observed temporal clustering exceeds shuffled networks, influence is present.

3.4 Expected End Product

1. **Quantification:** Decompose adoption into influence vs. homophily components
2. **Comparison:** Which tool shows strongest network effects? (Hypothesis: Claude Code, due to visible co-author attribution)
3. **Recommendations:** Evidence-based strategies for technology diffusion

4 Project Proposal 3: Multi-Tool Adoption as a Multiplex Network

4.1 The Idea

Core Question

How do developers navigate the ecosystem of competing AI tools, and does adoption of one tool predict adoption or rejection of others?

Rather than analyzing each tool in isolation, we model the entire AI coding ecosystem as a **multiplex network** with four layers:

4.2 Research Questions

1. **Cross-adoption:** Do developers who use Claude Code also use Copilot?
2. **Tool Migration:** Do developers switch from one tool to another over time?
3. **Ecosystem Dynamics:** Is the market trending toward winner-take-all or coexistence?
4. **Repository Heterogeneity:** Do repositories use multiple tools simultaneously?

4.3 Methodology

4.3.1 User Linking Across Tools

Match developers across datasets using:

- GitHub username (exact match)
- Repository co-occurrence (same repos with different tools)

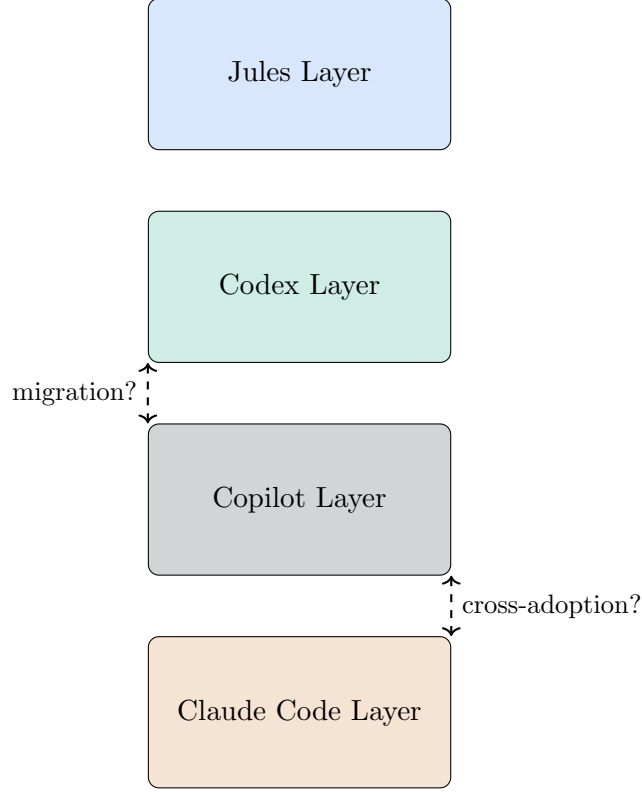


Figure 1: Multiplex network representation of AI tool ecosystem

4.3.2 Multiplex Network Metrics

- **Layer correlation:** Are the same developers central in multiple layers?
- **Multiplexity:** How many layers does each developer participate in?
- **Inter-layer community detection:** Do communities align across tools?

4.3.3 Transition Analysis

Model tool switching as a Markov process:

$$P(\text{Tool}_{t+1} = j \mid \text{Tool}_t = i) = \pi_{ij}$$

Estimate transition matrix Π from temporal adoption data.

4.4 Expected End Product

1. **Market Map:** Visualization of tool ecosystem with adoption flows
2. **Transition Matrix:** Quantified migration patterns between tools
3. **Predictions:** Future market share based on current trends
4. **Segmentation:** Developer archetypes (loyalists, explorers, pragmatists)

5 Project Proposal 4: Geographic Diffusion of AI Coding Tools

5.1 The Idea

Core Question

How does AI coding tool adoption spread geographically, and are there regional leader-follower dynamics in technology adoption?

Using the `owner_location` field in our repository metadata, we can analyze geographic patterns in AI tool adoption. This connects to literature on:

- Spatial diffusion of innovation
- Technology adoption by country
- Time zone effects on open source collaboration

5.2 Motivation

Geographic analysis can reveal:

1. Which regions are leading vs. lagging in AI tool adoption
2. Whether adoption spreads through geographic proximity
3. Cultural or regulatory factors affecting tool choice
4. Opportunities for localized developer outreach

5.3 Methodology

5.3.1 Location Extraction and Geocoding

1. Parse `owner_location` strings
2. Geocode to country/region using NLP
3. Handle ambiguous locations (“Bay Area”, “Remote”)

5.3.2 Spatial Network Construction

- Nodes = countries or regions
- Edges = collaboration intensity (developers from different regions working on same repos)
- Edge weights = number of cross-region collaborations

5.3.3 Spatial Diffusion Model

Adapt epidemic models for spatial spread:

$$\frac{dI_r}{dt} = \beta S_r \sum_{r'} w_{rr'} I_{r'} - \gamma I_r$$

where I_r is infected (adopted) proportion in region r , and $w_{rr'}$ is collaboration intensity between regions.

5.4 Expected End Product

1. **World Map:** Animated visualization of tool adoption spreading geographically
2. **Regional Analysis:** Adoption curves by country/continent
3. **Leader-Follower:** Identify which regions set trends vs. follow
4. **Comparison:** Do different tools have different geographic patterns?

6 Project Proposal 5: Influence Maximization for AI Tool Adoption

6.1 The Idea

Core Question

Which developers should an AI company target to maximize adoption through network effects?

This is the classic **influence maximization problem**: given a budget to recruit k “seed” developers, which k developers will lead to maximum total adoption?

6.2 Motivation

For AI coding tool companies, this has direct business value:

- Developer advocates and evangelists
- Open source sponsorship targets
- Conference speaker selection
- Beta program invitations

6.3 Methodology

6.3.1 Network Construction

Build collaboration network weighted by:

- Number of shared repositories
- Recency of collaboration
- Size/importance of shared repositories (stars)

6.3.2 Influence Maximization Algorithms

Implement and compare:

1. **Greedy algorithm:** Select node with maximum marginal influence gain
2. **CELF/CELF++:** Cost-effective lazy forward selection

3. **Degree heuristic:** Select highest-degree nodes
4. **PageRank heuristic:** Select highest PageRank nodes

6.3.3 Validation

Using historical data:

1. Identify actual early adopters (first 10% of adoptions)
2. Compare to algorithm-selected seeds
3. Measure downstream adoption from each seed set

6.4 Expected End Product

1. **Ranked Lists:** Top 100 influential developers for each tool
2. **Algorithm Comparison:** Which approach works best for this domain?
3. **ROI Analysis:** Expected adoption multiplier from seed selection
4. **Tool:** Reusable influence maximization implementation

7 Project Proposal 6: Community Detection and Tool Ecosystem Clustering

7.1 The Idea

Core Question

Do distinct communities form around different AI tools, and what characterizes these communities?

We hypothesize that developer communities may cluster by AI tool preference, programming language, or application domain. Understanding these communities helps predict adoption patterns.

7.2 Methodology

7.2.1 Bipartite Network Analysis

The natural structure is bipartite:

- U = set of developers
- V = set of repositories
- Edges = contributions

We can project this to:

- **Developer network:** Connect developers who share repositories
- **Repository network:** Connect repositories that share contributors

7.2.2 Community Detection

Apply multiple algorithms:

1. **Louvain:** Modularity optimization
2. **Infomap:** Information flow-based
3. **Label Propagation:** Scalable for large networks
4. **Spectral Clustering:** Eigenvector-based

7.2.3 Community Characterization

For each detected community, compute:

- Dominant programming language
- Common repository topics
- Geographic concentration
- AI tool preference distribution

7.3 Expected End Product

1. **Community Map:** Visualization of developer communities colored by tool preference
2. **Profiles:** Characterization of each major community
3. **Prediction:** Community membership as predictor of tool adoption
4. **Comparison:** How do communities differ across tools?

8 Summary: Recommended Project Selection

Table 2: Project Comparison Matrix

Project	Difficulty	Novelty	Impact	Feasibility
1. Cascade Effects	Medium	High	High	High
2. Influence vs. Homophily	High	Very High	Very High	Medium
3. Multiplex Network	Medium	High	Medium	High
4. Geographic Diffusion	Medium	Medium	Medium	High
5. Influence Maximization	Medium	Medium	High	High
6. Community Detection	Low-Medium	Medium	Medium	Very High

8.1 Our Recommended Selection

For a networks course project, we recommend:

1. Primary Project: Cascade Effects in AI Tool Adoption (Proposal 1)

- Strong connection to course material (information cascades)
- Novel application domain (AI coding tools)
- Rich temporal data enables rigorous analysis
- Produces compelling visualizations

2. Secondary Project: Social Influence vs. Homophily (Proposal 2)

- Addresses fundamental question in network science
- Methodologically sophisticated
- High potential for surprising findings
- Publishable research potential

9 Technical Appendix: Data Processing Pipeline

9.1 Network Construction Code Outline

```
# Build developer collaboration network
import pandas as pd
import networkx as nx

# Load commit data
commits = pd.read_parquet("claude_commits.parquet")

# Build repo -> developer mapping
repo_devs = commits.groupby("repo_nwo")["author_login"].apply(set)

# Build collaboration graph
G = nx.Graph()
for repo, devs in repo_devs.items():
    devs = list(devs)
    for i, d1 in enumerate(devs):
        for d2 in devs[i+1:]:
            if G.has_edge(d1, d2):
                G[d1][d2]["weight"] += 1
            else:
                G.add_edge(d1, d2, weight=1)
```

9.2 Cascade Detection Outline

```
# Detect adoption cascades
def detect_cascades(G, adoptions):
    """
```

```

adoptions: DataFrame with columns [user, timestamp]
Returns: Dict mapping each adopter to their likely "parent"
"""
cascades = {}
sorted_adoptions = adoptions.sort_values("timestamp")
adopted = set()

for _, row in sorted_adoptions.iterrows():
    user = row["user"]
    # Find neighbors who adopted before this user
    neighbors = set(G.neighbors(user)) & adopted
    if neighbors:
        # Assign to most recent adopter among neighbors
        parent = max(neighbors,
                     key=lambda n: adoptions[adoptions.user==n].timestamp.max())
        cascades[user] = parent
    else:
        cascades[user] = None # Seed node
    adopted.add(user)

return cascades

```

10 References and Related Work

Key papers to build upon:

1. Kempe, D., Kleinberg, J., & Tardos, E. (2003). Maximizing the spread of influence through a social network. *KDD*.
2. Aral, S., Muchnik, L., & Sundararajan, A. (2009). Distinguishing influence-based contagion from homophily-driven diffusion. *PNAS*.
3. Centola, D. (2010). The spread of behavior in an online social network experiment. *Science*.
4. Bakshy, E., et al. (2012). The role of social networks in information diffusion. *WWW*.
5. Leskovec, J., et al. (2007). The dynamics of viral marketing. *TWEB*.